TECHNICAL DOCUMENT
v1.0 · abril 2026
David Fdez · NyxSec

ARQUITECTURA · STACK · IMPLEMENTACIÓN

radio.gofestivals

Documento técnico de arquitectura y construcción

Especificación completa del sistema: stack, infraestructura, modelo de datos, API, pipeline de ingesta, servicio de metadata en vivo, frontend y observabilidad. De la decisión de diseño al código que se despliega.

FastAPI Next.js 15 PostgreSQL + PostGIS Redis Hetzner

21 páginas

00 Índice

Este documento especifica la arquitectura técnica de radio.gofestivals en detalle suficiente como para implementarlo sin ambigüedad. Cada decisión viene con su tradeoff explícito.

01	Resumen y alcance	Objetivos, no-objetivos, criterios de éxito	03
02	Vista de sistema	Diagrama C4 nivel contexto y contenedores	04
03	Stack tecnológico	Versiones, librerías, justificación de elecciones	05
04	Infraestructura	Hetzner, Docker, Traefik, DNS, secrets	06
05	Modelo de datos	Schema completo con DDL, índices, PostGIS	07
06	Backend · estructura	Layout del proyecto FastAPI, layers, DI	09
07	API · endpoints	Contratos REST y WebSocket	10
08	Pipeline Radio-Browser	Sync nocturno, taxonomía, health-checks	11
09	Servicio now-playing	Worker ICY, WebSocket, modo ambient	13
10	Frontend Next.js	App Router, data fetching, reproductor	14
11	Admin y curación	Auth, panel de revisión editorial	15
12	Observabilidad	Logs estructurados, Prometheus, alertas	16
13	CI/CD y testing	GitLab pipeline, pytest, Playwright	17
14	Desarrollo con Claude Code	CLAUDE.md, convenciones, reglas del agente, flujo	18
15	Checklist producción	Go/no-go para cada fase del roadmap	19

01 Resumen y alcance

Plataforma de descubrimiento de radio electrónica con curación editorial, metadata en vivo y vínculo con el catálogo de festivales existente. Subdominio dentro del ecosistema gofestivals, no producto separado.

Qué es

Una aplicación web que lista estaciones de radio de música electrónica, permite filtrarlas por género y país, reproducirlas en el navegador sin instalar nada, y ver en tiempo real qué suena en cada una. Las estaciones provienen de dos fuentes: un sync automático con Radio-Browser y una capa de curación manual que aprueba, rechaza y reclasifica cada entrada.

Qué no es

No es un agregador universal tipo radio.garden — ni pretende serlo. No incluye música no-electrónica. No hace descubrimiento geoespacial tipo globo 3D. No es una app nativa en fase inicial (PWA más adelante). No monetiza directamente; funciona como vehículo de retención para el ecosistema gofestivals.

Criterios de éxito técnicos

Latencia API p95	Menor a 200ms para listados filtrados, medida desde Europa
Time-to-first-sound	Menor a 3s desde click en play hasta primer buffer audible
Disponibilidad	99.5% mensual. Equivale a ~3.6h de downtime/mes
Catálogo inicial	Mínimo 300 estaciones tras primer sync; 500+ tras curación fase 2
Now-playing coverage	Metadata disponible en 60%+ de estaciones curadas

Restricciones y asunciones

RESTRICCIÓN · LEGAL

No se hace hosting de streams ni de cover art. Solo se indexan URLs públicas de terceros. Si una emisora solicita retirada, se elimina. No se aplican DMCA notifications propias ya que no se actúa como publisher.

ASUNCIÓN · RADIO-BROWSER

La dependencia de Radio-Browser para el sync inicial es aceptable: su infraestructura distribuida (pool de servidores resueltos por DNS SRV) es estable y su API no ha roto compatibilidad en años. El esfuerzo de reemplazar la fuente más adelante es bajo dado que solo se usa como seed.

TRADEOFF · CURACIÓN MANUAL

La curación manual tiene coste en tiempo (horas/semana). A cambio, la calidad del catálogo es lo que diferencia radio.gofestivals de un agregador más. Es decisión deliberada, no limitación.

02 Vista de sistema

Arquitectura en capas con separación clara entre frontend, API, datos y workers asíncronos. Un único binario FastAPI sirve el API; los workers son procesos separados que comparten base de datos pero no estado en memoria.

Contexto

Contenedores

```
┌──────────────────────────────────────────────────────────────┐
│  web-frontend       Next.js 15 · SSR + ISR · Tailwind        │
│  Puerto 3000 · renderiza listados, filtros, reproductor      │
└──────────────────────────┬───────────────────────────────────┘
                           │ HTTPS · JSON · WS
┌──────────────────────────▼───────────────────────────────────┐
│  api                FastAPI · uvicorn · async                │
│  Puerto 8000 · REST + WS · depende de Postgres y Redis       │
└──────┬──────────────────────────┬────────────────────────────┘
       │                          │
┌──────▼──────────┐  ┌────────────▼────────────┐  ┌──────────────┐
│ Postgres 16     │  │ Redis 7                 │  │ gofestivals  │
│ + PostGIS 3.4   │  │ cache + pub/sub + rate  │  │ MySQL 8      │
└──────▲──────────┘  └────────────▲────────────┘  └──────▲───────┘
       │                          │                     │
┌──────┴──────────┐  ┌────────────┴────────────┐  solo lectura
│ rb-sync         │  │ icy-worker              │  (view)
│ cron diario     │  │ python asyncio pool     │
│ 04:00 UTC       │  │ on-demand + ambient 60s │
└─────────────────┘  └─────────────────────────┘
```

DECISIÓN · MONOLITO MODULAR

Un solo proceso FastAPI sirve todas las rutas (públicas, admin, websocket). No hay microservicios. El único proceso que podría separarse si hace falta es icy-worker, porque tiene perfil de recursos distinto (conexiones persistentes). Para el volumen previsto en fases 1 y 2, un monolito es óptimo.

03 Stack

Elecciones conservadoras en tecnologías probadas. Sin novedades por novedad. Cada pieza se justifica por problema que resuelve, no por trendiness.

Frontend

PIEZA	VERSIÓN	POR QUÉ
Next.js	15.x	App Router + ISR. SEO necesario para captar tráfico de búsquedas por género
React	19.x	Stack estándar, compatibilidad máxima
TypeScript	5.x	Tipado en los contratos con API evita regresiones
Tailwind	4.x	Coherencia con el design system del brand doc
shadcn/ui	latest	Componentes base sin lock-in. Copiables y editables
Zustand	5.x	Estado cliente mínimo: player global, preferencias
TanStack Query	5.x	Cache y refetch de datos de API, no reimplementarlo

Backend

PIEZA	VERSIÓN	POR QUÉ
Python	3.12	Versión estable actual, async maduro
FastAPI	0.115+	Async nativo, OpenAPI gratis, DI buena
uvicorn	latest	ASGI server. Detrás de Traefik, no necesita gunicorn
SQLAlchemy	2.x async	ORM probado, queries tipadas, async nativo
asyncpg	latest	Driver Postgres async. Más rápido que psycopg3 en bulk
Alembic	1.14+	Migraciones. Estándar del ecosistema SQLAlchemy
redis-py	5.x	Cache y pub/sub
httpx	latest	Cliente HTTP async para Radio-Browser e ICY
@music-metadata/icy	latest	Parser ICY probado (desde Node, vía subprocess o portear)
structlog	25.x	Logs estructurados JSON para Loki/Grafana

Datos e infraestructura

PIEZA	VERSIÓN	POR QUÉ
PostgreSQL	16	Base de datos principal. Estable, maduro, PostGIS encaja

04 Infraestructura

VPS Hetzner con Docker Compose detrás de Traefik. Modelo idéntico al que ya gestionas en tus otros proyectos. Un único host en fase 1; multi-host solo si el tráfico lo justifica.

Topología

VPS Hetzner	CX22 inicial (2 vCPU, 4GB RAM, 40GB NVMe). Upgrade a CX32 si p95 degrada
OS	Ubuntu 24.04 LTS. Mismo base image que el resto de tu parque
DNS	Registro A para `radio.gofestivals.es` y `radio.gofestivals.eu`
TLS	Traefik + Let's Encrypt HTTP-01. Renovación automática
Firewall	UFW. Abre 22, 80, 443. Resto denegado
Secrets	`.env` en host, nunca en repo. `docker-compose.yml` los lee

docker-compose.yml

```yaml
version: "3.9"

services:
  api:
    image: registry.gitlab.com/gofestivals/radio-api:${TAG:-latest}
    restart: unless-stopped
    env_file: .env
    depends_on: [postgres, redis]
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.api.rule=Host(`api.radio.gofestivals.es`)"
      - "traefik.http.routers.api.tls.certresolver=le"
      - "traefik.http.services.api.loadbalancer.server.port=8000"

  web:
    image: registry.gitlab.com/gofestivals/radio-web:${TAG:-latest}
    restart: unless-stopped
    env_file: .env
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.web.rule=Host(`radio.gofestivals.es`)"
      - "traefik.http.routers.web.tls.certresolver=le"
      - "traefik.http.services.web.loadbalancer.server.port=3000"

  icy-worker:
    image: registry.gitlab.com/gofestivals/radio-api:${TAG:-latest}
    restart: unless-stopped
    command: python -m app.workers.icy
    env_file: .env
    depends_on: [postgres, redis]

  postgres:
    image: postgis/postgis:16-3.4
    restart: unless-stopped
    environment:
      POSTGRES_DB: radiogf
      POSTGRES_USER: ${PGUSER}
      POSTGRES_PASSWORD: ${PGPASSWORD}
    volumes: ["pgdata:/var/lib/postgresql/data"]

  redis:
    image: redis:7-alpine
    restart: unless-stopped
```

05 Modelo de datos

Cinco tablas nucleares. La taxonomía de géneros está normalizada en tabla propia para desacoplarse de los tags sucios de Radio-Browser. Coordenadas como `geography(POINT)` de PostGIS para queries por distancia.

DDL · extensiones y tipos

```
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS pg_trgm;  -- búsqueda por nombre

CREATE TYPE station_status AS ENUM (
  'pending', 'active', 'broken', 'rejected'
);
CREATE TYPE festival_link_type AS ENUM (
  'official', 'city', 'lineup'
);
```

DDL · stations

```
CREATE TABLE stations (
  id            uuid        PRIMARY KEY DEFAULT uuid_generate_v4(),
  rb_uuid       uuid        UNIQUE,                  -- stationuuid de Radio-Browser
  slug          text        UNIQUE NOT NULL,
  name          text        NOT NULL,
  stream_url    text        NOT NULL,
  homepage_url  text,
  country_code  char(2),
  city          text,
  geo           geography(POINT, 4326),
  codec         text,                            -- mp3, aac, opus
  bitrate       int,                             -- kbps
  language      text,
  curated       bool        NOT NULL DEFAULT false,
  quality_score smallint    NOT NULL DEFAULT 50,    -- 0-100
  status        station_status NOT NULL DEFAULT 'pending',
  failed_checks smallint    NOT NULL DEFAULT 0,
  last_check_ok timestamptz,
  last_sync_at  timestamptz,
  created_at    timestamptz NOT NULL DEFAULT now(),
  updated_at    timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX idx_stations_status        ON stations(status);
CREATE INDEX idx_stations_curated_status ON stations(curated, status)
  WHERE status = 'active';
CREATE INDEX idx_stations_geo            ON stations USING gist(geo);
CREATE INDEX idx_stations_name_trgm      ON stations
  USING gin(name gin_trgm_ops);
```

DETALLE · GEO COMO GEOGRAPHY

Se usa `geography(POINT, 4326)` en vez de `geometry` porque las queries son por distancia en km (no por área). Geography usa esfera, `ST_DWithin` devuelve resultados correctos sin conversiones.

DDL · genres y station_genres

```
CREATE TABLE genres (
  id         smallserial PRIMARY KEY,
  slug       text        UNIQUE NOT NULL,       -- 'techno', 'deep-house'
  name       text        NOT NULL,
  parent_id  smallint    REFERENCES genres(id),
  color_hex  char(7)     NOT NULL DEFAULT '#8B4EE8',
  sort_order smallint    NOT NULL DEFAULT 100
);

CREATE TABLE station_genres (
  station_id uuid     REFERENCES stations(id) ON DELETE CASCADE,
  genre_id   smallint REFERENCES genres(id),
  confidence smallint NOT NULL DEFAULT 50,     -- 0-100
  source     text     NOT NULL,                 -- 'rb-tag', 'manual', 'rule'
  PRIMARY KEY (station_id, genre_id)
);

CREATE INDEX idx_station_genres_genre ON station_genres(genre_id);
```

DDL · now_playing

```
CREATE TABLE now_playing (
  id           bigserial   PRIMARY KEY,
  station_id   uuid        NOT NULL REFERENCES stations(id) ON DELETE CASCADE,
  title        text        NOT NULL,
  artist       text,
  raw_metadata text,
  captured_at  timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX idx_np_station_time ON now_playing(station_id, captured_at DESC);
-- Retención: borrado semanal de > 30 días via pg_cron
```

DDL · festival_stations

Vínculo con la tabla `festivals` de gofestivals. Vive en MySQL (WordPress), así que no hay FK: se usa el ID como valor y la API une en memoria cuando hace falta. Más simple que sincronizar catálogos entre bases.

```
CREATE TABLE festival_stations (
  id          bigserial         PRIMARY KEY,
  festival_id bigint            NOT NULL,    -- FK lógica a gofestivals.festivals.id
  station_id  uuid              NOT NULL REFERENCES stations(id) ON DELETE CASCADE,
  link_type   festival_link_type NOT NULL,
  notes       text,
  created_at  timestamptz       NOT NULL DEFAULT now(),
  UNIQUE (festival_id, station_id, link_type)
);
CREATE INDEX idx_fs_station  ON festival_stations(station_id);
CREATE INDEX idx_fs_festival ON festival_stations(festival_id);
```

SEED · GÉNEROS ELECTRÓNICOS

Taxonomía inicial de 25 géneros raíz con jerarquía de dos niveles, insertada en `alembic/versions/0002_genres_seed.py`: techno (+ minimal, hard), house (+ deep, tech, progressive), trance, dnb, dubstep, ambient, hardstyle, breakbeat, electro, idm, garage, jungle y más. Cada género con su `color_hex` para la UI.

06 Backend

Layout del proyecto FastAPI organizado por capas. Separación entre rutas, servicios, repos y modelos. Dependency injection vía `Depends` nativo de FastAPI, sin frameworks de DI externos.

Árbol del repo

```
radio-api/
├── pyproject.toml
├── alembic.ini
├── alembic/
│   └── versions/
├── app/
│   ├── main.py             # FastAPI app factory
│   ├── config.py           # pydantic-settings
│   ├── deps.py             # DI: db, redis, current_user
│   ├── api/
│   │   ├── v1/
│   │   │   ├── stations.py   # GET listado y detalle
│   │   │   ├── genres.py     # GET árbol taxonomía
│   │   │   ├── now_playing.py# WS /ws/nowplaying/:slug
│   │   │   ├── festivals.py  # vínculo cross-db
│   │   │   └── admin.py      # rutas protegidas
│   ├── core/
│   │   ├── security.py       # JWT admin, hash passwords
│   │   ├── logging.py        # structlog config
│   │   └── cache.py          # helpers Redis
│   ├── db/
│   │   ├── session.py        # async engine y sessionmaker
│   │   └── base.py           # DeclarativeBase
│   ├── models/
│   │   ├── station.py
│   │   ├── genre.py
│   │   └── now_playing.py
│   ├── schemas/            # pydantic IO schemas
│   │   ├── station.py
│   │   └── genre.py
│   ├── repos/              # queries encapsuladas
│   │   ├── station.py
│   │   └── now_playing.py
│   ├── services/           # lógica de negocio
│   │   ├── curation.py
│   │   └── taxonomy_mapper.py
│   ├── jobs/               # CLI ejecutables por cron
│   │   ├── rb_sync.py
│   │   └── health_check.py
│   └── workers/            # procesos long-running
│       └── icy.py           # ambient + on-demand pool
└── tests/
    ├── unit/
    ├── integration/
    └── e2e/
```

Separación de capas

api/	Controladores HTTP/WS. Validación de request, llamada a servicio, serialización de response. Cero lógica de negocio aquí

07 API · endpoints

REST versionado bajo `/api/v1`. WebSocket para now-playing. Todo async. OpenAPI schema se autogenera y se expone en `/docs` solo en staging.

Públicos

GET `/api/v1/genres`

Árbol jerárquico de géneros con conteo de estaciones activas. Cacheado 5 min en Redis

GET `/api/v1/stations`

Listado paginado. Query params: `genre`, `country`, `curated`, `q` (search), `page`, `size` (max 50)

GET `/api/v1/stations/:slug`

Detalle de estación incluyendo géneros, festival-links y últimas 10 tracks

GET `/api/v1/stations/:slug/stream`

Redirige 302 al `stream_url`. Permite logging y rate-limiting sin poner el URL real en el frontend

GET `/api/v1/stations/:slug/nowplaying`

Última canción conocida + historial de 20. Útil para SSR inicial antes del WS

WS `/ws/nowplaying/:slug`

Suscripción a actualizaciones en vivo vía Redis pub/sub. Msg JSON `{title, artist, at}`

GET `/api/v1/festivals/:id/stations`

Radios vinculadas a un festival, agrupadas por `link_type`

GET `/api/v1/stations/nearby`

Params: `lat`, `lng`, `radius_km`. PostGIS `ST_DWithin`

Admin (JWT requerido)

POST `/api/v1/admin/auth/login`

Devuelve JWT. Single-user bootstrap, OAuth cuando haya equipo

GET `/api/v1/admin/stations/pending`

Cola de curación: estaciones nuevas sin revisar

PUT `/api/v1/admin/stations/:id/curate`

Body `{decision, genre_ids[], quality_score, notes}`. Marca curated y cambia status

08 Pipeline Radio-Browser

Sync idempotente diario. Resuelve SRV, itera la whitelist de tags electrónicos, dedupe por stationuuid, upsert. Mapea tags sucios a la taxonomía propia. Salud cada 6h marca broken tras tres fallos consecutivos.

Whitelist de tags

```
ELECTRONIC_TAGS = [
    "techno", "house", "deep house", "tech house",
    "minimal", "trance", "progressive", "drum and bass",
    "dnb", "dubstep", "ambient", "hardstyle",
    "breakbeat", "electronic", "edm",
]

# Mapeo tag sucio → genre slug canónico
TAG_ALIASES = {
    "drum and bass": "dnb",
    "d&b":           "dnb",
    "dn&b":          "dnb",
    "deephouse":      "deep-house",
    "tech-house":     "tech-house",
    "techhouse":      "tech-house",
    # ...
}
```

rb_sync.py · primitivas

```
import asyncio, logging
from dns import resolver
import httpx
from app.db.session import AsyncSessionLocal
from app.repos.station import StationRepo
from app.services.taxonomy_mapper import map_tags_to_genres

log = logging.getLogger("rb_sync")

async def resolve_rb_host() -> str:
    # Pool distribuido: resolvemos SRV, usamos el primero (priority/weight lo ordena)
    answers = resolver.resolve("_api._tcp.radio-browser.info", "SRV")
    return f"https://{str(answers[0].target).rstrip('.')}"

async def fetch_stations_by_tag(client, base, tag):
    resp = await client.get(
        f"{base}/json/stations/search",
        params={"tag": tag, "hidebroken": "true",
                "is_https": "true", "limit": 500},
        headers={"User-Agent": "radio.gofestivals/1.0"},
    )
    resp.raise_for_status()
    return resp.json()
```

La orquestación `run()` itera `ELECTRONIC_TAGS`, dedupe por `stationuuid`, y pasa cada item a `StationRepo.upsert_from_rb()`. El upsert y el resto del pipeline en la siguiente página.

Upsert con SQLAlchemy 2.x

```
from sqlalchemy.dialects.postgresql import insert
from app.models.station import Station

class StationRepo:
    def __init__(self, db): self.db = db

    async def upsert_from_rb(self, item: dict, genre_ids: list[int]):
        values = {
            "rb_uuid": item["stationuuid"],
            "name": item["name"].strip(),
            "slug": slugify(item["name"]),
            "stream_url": item["url_resolved"] or item["url"],
            "country_code": (item.get("countrycode") or "")[:2] or None,
            "codec": (item.get("codec") or "").lower() or None,
            "bitrate": item.get("bitrate") or None,
            "last_sync_at": datetime.now(tz=UTC),
        }
        stmt = insert(Station).values(**values).on_conflict_do_update(
            index_elements=["rb_uuid"],
            set_={k: "EXCLUDED."+k for k in values if k != "slug"},
        ).returning(Station.id)
        station_id = (await self.db.execute(stmt)).scalar_one()

        for gid in genre_ids:
            await self.db.execute(
                """INSERT INTO station_genres (station_id, genre_id, source, confidence)
                   VALUES (:s, :g, 'rb-tag', 60) ON CONFLICT DO NOTHING""",
                {"s": station_id, "g": gid},
            )
```

Health check · cada 6h

```
async def check_station(client, station):
    try:
        r = await client.head(station.stream_url, timeout=5,
                              follow_redirects=True)
        ok = r.status_code < 400
    except Exception:
        ok = False

    if ok:
        station.failed_checks = 0
        station.last_check_ok = datetime.now(tz=UTC)
        if station.status == "broken":
            station.status = "active"
    else:
        station.failed_checks += 1
        if station.failed_checks >= 3:
            station.status = "broken"
```

Mapeo tag → taxonomía

El mapeo canónico evita que dos estaciones con tags "drum-and-bass" y "dnb" queden en categorías distintas. Algoritmo: normalizar (lower + trim + collapse spaces), aplicar aliases, buscar match en `genres.slug`, ignorar lo que no case. El resto queda como `source='rb-tag'` con `confidence=60` hasta que el curador confirma a `'manual'` / `100`.

> **IDEMPOTENCIA**
>
> El job se puede ejecutar N veces sin duplicar datos. Si cambia cualquier campo en Radio-Browser (ej. URL resolved) se actualiza en el upsert. Los campos controlados por curación manual (`quality_score`, `curated`) no se pisan porque no aparecen en el `set_` del upsert.

09 Servicio now-playing

Worker dedicado que parsea ICY metadata de streams Icecast/Shoutcast. Dos modos: **on-demand** cuando alguien reproduce, **ambient** polleando top curadas. Publica en Redis pub/sub, el WS del API reenvía al cliente.

Arquitectura del worker

```
icy-worker (proceso Python, asyncio)
  │
  ├─ ambient loop (cada 60s)
  │    SELECT stations WHERE curated=true AND status=active
  │          ORDER BY quality_score DESC LIMIT 50
  │    for each → IcecastMetadataStats.get()  (HEAD con Icy-MetaData)
  │    if changed → insert now_playing + publish Redis
  │
  └─ on-demand pool (WS triggered)
       Redis SUB "play:*"   ← emite API cuando llega cliente a WS
       spawn task por station si no existe
       conectar con Icy-MetaData:1, leer stream chunks
       extraer StreamTitle cada intervalo
       cleanup cuando último subscriber cierra
```

Trigger on-demand desde API

```python
# app/api/v1/now_playing.py
@router.websocket("/ws/nowplaying/{slug}")
async def nowplaying_ws(ws: WebSocket, slug: str,
                        db=Depends(get_db), r=Depends(get_redis)):
    station = await StationRepo(db).get_by_slug(slug)
    if not station: await ws.close(code=4404); return

    await ws.accept()

    # Aviso al worker: alguien escuchando, activa on-demand
    await r.publish("play:request", str(station.id))

    pubsub = r.pubsub()
    await pubsub.subscribe(f"np:{station.id}")

    try:
        async for msg in pubsub.listen():
            if msg["type"] == "message":
                await ws.send_text(msg["data"])
    except WebSocketDisconnect:
        await r.publish("play:release", str(station.id))
    finally:
        await pubsub.unsubscribe()
```

Extracción ICY

```python
async def read_icy(url: str, station_id: UUID, r: Redis):
    headers = {"Icy-MetaData": "1",
               "User-Agent": "radio.gofestivals/1.0"}

    async with httpx.AsyncClient(timeout=None) as client:
        async with client.stream("GET", url, headers=headers) as resp:
            interval = int(resp.headers.get("icy-metaint", 0))
            if not interval:
                log.info("no_icy", url=url)
                return
```

10 Frontend

App Router con ISR para páginas de género (buenas para SEO). Reproductor HTML5 global con Zustand. TanStack Query para listados. Sin state managers pesados.

Estructura del repo

```
radio-web/
├── next.config.ts
├── tailwind.config.ts
├── src/
│   ├── app/
│   │   ├── layout.tsx         # shell + player global
│   │   ├── page.tsx           # home · destacados
│   │   ├── [genre]/           # /techno, /house, ...
│   │   │   └── page.tsx       # ISR · revalidate 300
│   │   ├── stations/[slug]/
│   │   │   └── page.tsx       # detalle estación
│   │   ├── festivales/[id]/
│   │   │   └── page.tsx       # radios de un festival
│   │   └── api/
│   │       └── proxy/          # proxies al backend si hace falta
│   ├── components/
│   │   ├── StationGrid.tsx
│   │   ├── StationCard.tsx
│   │   ├── GenreSidebar.tsx
│   │   ├── player/
│   │   │   ├── GlobalPlayer.tsx   # bottom sticky
│   │   │   └── useAudioEngine.ts  # HTMLAudioElement ref
│   ├── lib/
│   │   ├── api.ts             # fetch wrappers tipados
│   │   └── nowplaying-ws.ts   # hook WS auto-reconnect
│   └── store/
│       └── player.ts          # Zustand: currentStation, state
```

Store del reproductor (Zustand)

```
import { create } from "zustand";
import { persist } from "zustand/middleware";

type PlayerState = {
  current: { slug: string; name: string; streamUrl: string } | null;
  status: "idle" | "loading" | "playing" | "paused" | "error";
  volume: number;
  nowPlaying: { title: string; artist?: string } | null;
  play: (station: NonNullable<PlayerState["current"]>) => void;
  stop: () => void;
  setNowPlaying: (np: PlayerState["nowPlaying"]) => void;
};

export const usePlayer = create<PlayerState>()(
  persist(
    (set) => ({
      current: null,
      status: "idle",
      volume: 0.8,
      nowPlaying: null,
      play: (station) => set({ current: station, status: "loading", nowPlaying: null }),
      stop: () => set({ current: null, status: "idle", nowPlaying: null }),
      setNowPlaying: (np) => set({ nowPlaying: np }),
    }),
    { name: "rgf-player", partialize: (s) => ({ volume: s.volume }) },
```

11 Admin y curación

Panel separado detrás de `/admin` en el frontend. Auth JWT simple. Objetivo: revisar rápido (target ~30 seg/estación) la cola de pending y clasificar.

Auth

Usuario único al arrancar, definido vía `ADMIN_EMAIL` y hash bcrypt en `ADMIN_PASSWORD_HASH`. Cuando haya más gente se migra a tabla `users` con roles. Fase 1 es single-admin deliberadamente.

```
from datetime import datetime, timedelta, UTC
from jose import jwt
import bcrypt

async def login(email: str, password: str) -> str:
    if email != settings.ADMIN_EMAIL:
        raise HTTPException(401, "invalid credentials")
    if not bcrypt.checkpw(password.encode(),
                          settings.ADMIN_PASSWORD_HASH.encode()):
        raise HTTPException(401, "invalid credentials")
    payload = {
        "sub": email,
        "exp": datetime.now(UTC) + timedelta(hours=12),
        "role": "admin",
    }
    return jwt.encode(payload, settings.JWT_SECRET, algorithm="HS256")
```

Cola de curación · UI

Lista las estaciones `status='pending'` ordenadas por fecha de sync. Cada una muestra: reproductor embebido para escuchar 10-15s, tags originales de Radio-Browser, sugerencia auto-asignada de géneros, campos editables. Atajos de teclado: `A` aprobar, `R` rechazar, `1-9` asignar géneros, `Enter` siguiente.

Decisión de curación

```
class CurationDecision(BaseModel):
    decision: Literal["approve", "reject"]
    genre_ids: list[int] = []
    quality_score: int = 50  # 0-100
    notes: str | None = None

async def apply_curation(db, station_id: UUID, d: CurationDecision):
    station = await StationRepo(db).get(station_id)
    if not station: raise HTTPException(404)

    if d.decision == "reject":
        station.status = "rejected"
        station.curated = True
        await db.commit()
        return

    station.status = "active"
    station.curated = True
    station.quality_score = d.quality_score

    # reemplazar géneros con los confirmados manualmente
    await db.execute(
        "DELETE FROM station_genres WHERE station_id = :s AND source = 'rb-tag'",
        {"s": station_id},
    )
    for gid in d.genre_ids:
        await db.execute(
            "INSERT INTO station_genres (station_id, genre_id, source, confidence) "
            "VALUES (:s, :g, 'manual', 100) ON CONFLICT DO NOTHING",
            {"s": station_id, "g": gid},
        )
```

12 Observabilidad

Logs estructurados JSON enviados a Loki. Métricas Prometheus nativas de FastAPI. Dashboards Grafana en tu stack existente. Alertas por Telegram al bot que ya usas para Nyx.

Logs estructurados

```
# app/core/logging.py
import structlog, logging, sys

def configure_logging(level="INFO"):
    logging.basicConfig(format="%(message)s", stream=sys.stdout, level=level)
    structlog.configure(
        processors=[
            structlog.contextvars.merge_contextvars,
            structlog.processors.add_log_level,
            structlog.processors.TimeStamper(fmt="iso"),
            structlog.processors.StackInfoRenderer(),
            structlog.processors.format_exc_info,
            structlog.processors.JSONRenderer(),
        ],
        wrapper_class=structlog.make_filtering_bound_logger(logging.INFO),
        logger_factory=structlog.stdlib.LoggerFactory(),
        cache_logger_on_first_use=True,
    )
```

Métricas Prometheus

```
from prometheus_fastapi_instrumentator import Instrumentator

# Métricas nativas: request count, latency histograms, in-progress
Instrumentator().instrument(app).expose(app, endpoint="/metrics")

# Custom metrics para el worker ICY
from prometheus_client import Counter, Gauge
icy_extractions = Counter("icy_extractions_total", "ICY titles extracted",
                          ["status"])
icy_active = Gauge("icy_workers_active", "Active ICY workers")
```

Dashboards clave

01

API health

Request rate, p50/p95/p99 latency, error rate por endpoint. SLO: p95 < 200ms.

02

Catálogo

Estaciones por status, ratio curadas, pending backlog, new sync deltas.

03

Now-playing

Workers activos, extractions/min, cobertura (% estaciones con metadata hoy).

04

Traffic

Top géneros, top estaciones, hours distribution, concurrent listeners peak.

Alertas

api_down	`up{job="radio-api"} == 0` por 2 min → Telegram crítico

13 CI/CD y testing

Pipeline GitLab en cinco fases: lint, test, build, security, deploy. Promotion a producción manual, staging automático en merge a `main`. Tests en tres niveles: unit, integration con DB real, e2e con Playwright.

.gitlab-ci.yml

```yaml
stages: [lint, test, build, security, deploy]

.py-base: &py-base
  image: python:3.12-slim
  cache:
    key: pip-${CI_COMMIT_REF_SLUG}
    paths: [.cache/pip]

lint:
  <<: *py-base
  stage: lint
  script:
    - pip install ruff mypy
    - ruff check app
    - mypy app --strict

test-unit:
  <<: *py-base
  stage: test
  script: [pip install -e .[test], pytest tests/unit -v]

test-integration:
  <<: *py-base
  stage: test
  services: [postgis/postgis:16-3.4, redis:7]
  variables:
    DATABASE_URL: "postgresql+asyncpg://postgres:postgres@postgis/test"
  script: [pip install -e .[test], alembic upgrade head, pytest tests/integration -v]

build:
  stage: build
  image: docker:24
  services: [docker:24-dind]
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA .
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

sast:
  stage: security
  image: returntocorp/semgrep
  script: [semgrep --config=auto --error app/]

deploy-staging:
  stage: deploy
  only: [main]
  script:
    - ssh deploy@staging "cd /opt/radio && TAG=$CI_COMMIT_SHORT_SHA docker compose up -d"

deploy-prod:
  stage: deploy
  when: manual
  only: [main]
  script:
    - ssh deploy@prod "cd /opt/radio && TAG=$CI_COMMIT_SHORT_SHA docker compose up -d"
```

Testing strategy

U

Unit

I

Integration

E

E2E

14 Claude Code

Este proyecto se desarrolla usando Claude Code como par de programación. El repo incluye convenciones explícitas y un `CLAUDE.md` en raíz que da contexto al agente en cada sesión. Lo que sigue son las reglas del juego.

CLAUDE.md · contexto raíz

Todo repo tiene un `CLAUDE.md` en raíz. Es lo primero que Claude Code lee al abrir el proyecto. Contiene objetivos, stack, estructura, comandos y convenciones. Cuando cambian, se actualiza antes que el código que rompe.

```
radio.gofestivals/
├── CLAUDE.md          ← contexto raíz (leído siempre)
├── README.md
├── docker-compose.yml
├── .env.example
├── apps/
│   ├── api/            ← FastAPI (Python 3.12)
│   │   ├── CLAUDE.md    ← contexto específico del backend
│   │   └── app/
│   └── web/            ← Next.js 15 (Node 20)
│       ├── CLAUDE.md    ← contexto específico del frontend
│       └── src/
├── packages/
│   ├── icy-worker/     ← worker Python de metadata
│   └── scripts/        ← sync, taxonomy mapper, maintenance
└── infra/
    ├── traefik/
    ├── prometheus/
    └── migrations/     ← alembic
```

Reglas para el agente

No inventar API	Radio-Browser, PostGIS, ICY metadata: verificar antes con docs o ejemplo real. Prohibido asumir endpoints o firmas.
Tests antes de "done"	Sin test pasando = no terminado. Unit + integration para cambios en API, pipeline o DB. E2E solo para flujos críticos.
Migraciones atómicas	Cada cambio de schema → alembic revision. Nunca `alter` manual. Migrations reversibles cuando sea posible.
Commits pequeños	Un cambio lógico por commit. Subject <72 chars, imperativo. Scope prefix: `api:`, `web:`, `worker:`, `infra:`.
Sin secrets en código	Todo va por `.env`. Si falta una variable, falla ruidoso al arranque. Nunca commitear keys ni aún de "solo dev".
Decisiones → ADR	Decisiones no triviales se escriben en `docs/adr/NNN-titulo.md`. Formato corto: contexto, decisión, consecuencias.

Flujo de trabajo típico

```
# 1. Levantar stack local
$ cp .env.example .env
$ docker compose up -d postgres redis
$ cd apps/api && uv sync && alembic upgrade head

# 2. Decirle a Claude Code qué hacer
$ claude "Implementa el endpoint GET /api/v1/genres con caché Redis 5min"

# 3. El agente:
  · Lee CLAUDE.md (contexto)
  · Crea app/api/v1/genres.py siguiendo convenciones del repo
  · Añade test en tests/integration/test_genres.py
  · Ejecuta pytest + ruff + mypy
  · Si todo pasa, propone commit

# 4. Code review humano antes de merge a main
$ git diff main...HEAD
$ # Revisar: seguridad, performance, consistencia de naming
```

Tareas bien delimitadas · ejemplos

Buenas instrucciones son concretas, verificables y con criterio de éxito. Malas son vagas y abiertas. Ejemplos reales del proyecto:

✓ BUENA

"Implementa `rb_sync.py` siguiendo la spec de sección 08. Debe ser idempotente, respetar el whitelist de tags y mapear por `TAG_ALIASES`. Test: tras dos ejecuciones seguidas, `SELECT count(*) FROM stations` es igual."

✗ MALA

"Hazme el script de sincronización con Radio-Browser". Sin criterio de éxito, sin referencia al whitelist, sin mención del mapping. Resultado: código plausible pero que no encaja con la arquitectura.

15 Checklist producción

Go/no-go por fase del roadmap. No hay que cumplir todo para lanzar pero cada item sin marcar es una decisión explícita, no un descuido.

Fase 01 · MVP técnico GO

Infra	VPS provisioned, DNS apuntando, Traefik con TLS, .env con secrets rotables
DB	Postgres+PostGIS levantado, migraciones ejecutadas, seed de géneros insertado
API	Endpoints públicos funcionando: `/genres`, `/stations`, `/stations/:slug`, `/stream`
Pipeline	Primer sync manual OK, cron diario activado y verificado
Frontend	Home, página de género ISR, detalle, reproductor global mínimo
Smoke	300+ estaciones en status='active' tras el primer sync y cleanup inicial

Fase 02 · Diferenciadores EN CURSO

Admin	Login JWT, UI de cola pending, endpoint curate, atajos de teclado
Now-playing	Worker ICY funcionando, WS en frontend, cobertura > 60% top 50
Festivales	Vinculación cross-db, página `/festivales/[id]` listando radios
Curación	Backlog < 50 estaciones pending de forma sostenida

Fase 03 · Pulir y crecer CONTINUO

Observabilidad	Dashboards Grafana, alertas Telegram activas, SLO 99.5% trackeable
Analytics	Eventos play/stop con station_id, dashboards de uso
SEO	Páginas estáticas ISR para cada género, sitemap.xml, metadata OG
PWA	manifest.json, service worker, instalable en móvil
Perf	Lighthouse > 90 en mobile, TTFB < 500ms en páginas ISR

Riesgos operacionales conocidos

R1 · STREAMS QUE DESAPARECEN

Muchas emisoras pequeñas cambian URL o se caen sin aviso. Mitigación: health-check cada 6h + status='broken' + backoff exponencial en reintentos. Catálogo limpio requiere esfuerzo continuo.

R2 · CORS Y FORMATOS RAROS

Algunas estaciones sirven HLS en vez de mp3/aac directo. El reproductor HTML5 maneja mp3/aac/ogg pero no HLS sin hls.js. Decisión fase 1: excluir HLS del catálogo. Fase 2 considerar soporte si hay volumen suficiente.

R3 · RADIO-BROWSER OUTAGE

Si el sync falla N días seguidos no bloquea la app (datos ya cachean en DB). Pero si dura semanas, el catálogo se estanca. Mitigación: segundo proveedor como fallback, o aceptar que es un riesgo bajo dado el track record del servicio.